ICT DIGITAL START 2026 · FINALS · NOV 6 · OPEN SPACE · 14:45

Meal*Mate* — Finals Pack

Речь (2:00, EN) · Демо-скрипт · Шпаргалки каждого участника · Чек-лист дня.

Файл открывается на телефоне в браузере; для печати — PDF-версия.

CS-2618 · Samarin · Kaniyazov · Sadybek · Temirtas

0 · Пять цифр, которые знает каждый

Цифра	Что значит	Если спросят глубже
76 kg	выброшенной еды на человека в год в Казахстане	UNEP Food Waste Index Report 2021, household fraction
68 / 51 / 0	рецептов · ингредиентов в пикере · «мёртвых» ингредиентов (было 2)	аудит покрытия — Assignment 4, workbook с формулами
3	языка: интерфейс, контент и шаги рецептов — EN/РУ/ҚАЗ	локализованные колонки в БД, API ?lang= с фолбэком на EN
244	студента в опросе продовольственной безопасности (Алматы) — доказательство проблемы	исследование связало food security и well-being студентов
3 → 3	UX-проблемы из тестирования → все исправлены	выбор ингредиентов, явный финиш, кнопка «назад»

1 · Team Speech — 2:00 (English)

Говорить по-английски; русский перевод — чтобы понимать, ЧТО говоришь. Секвенция: Артур → Артур → Арсен → Адилет → Санжар → Артур.

0:00–0:20 **Artur — Intro + Problem**

"Hello! We are Team MealMate from group CS-2618. Here is a number for you: **76**

kilograms — the food one person in Kazakhstan throws away every year, according to

the UNEP Food Waste Index. The problem is that students living away from home

struggle to decide what to cook from the random products in their fridge — so they

waste food, waste money, and skip meals."

«Привет! Мы — команда MealMate, группа CS-2618. Цифра: 76 кг еды в год выбрасывает один

человек в Казахстане — данные UNEP. Проблема: студенты не решают, что приготовить из

случайных продуктов — еда и деньги на ветер».)

0:20–0:45 **Artur — Evidence**

"To confirm this is real, we studied the UNEP 2021 report and a food-security survey of

244 students in Almaty — the link between food security and student well-being is

proven. We also tested our prototype with real users: every single tester found a recipe

with our matcher in under two minutes, and told us what to improve — and we did."

«Подтверждение: UNEP 2021 + опрос 244 студентов в Алматы. Свои тесты: каждый

тестировщик нашёл рецепт меньше чем за 2 минуты — и мы исправили всё, что они

отметили».)

0:45–1:10 **Arsen — Solution**

"Our solution is MealMate — a web app that matches recipes to the food you already

have. You tick your ingredients, and it instantly splits **68 recipes** into 'cook right now'

and 'almost there' — showing exactly what to buy. What makes it different: it is built for

local kitchens — **Kazakh cuisine is first-class** — and it works in three languages:

English, Russian and Kazakh."

«MealMate сопоставляет рецепты с тем, что есть. Отмечаешь продукты — 68 рецептов

делятся на „готовить сейчас" и „почти готово" с точным списком покупок. Отличие:

казахская кухня first-class, три языка».)

1:10–1:30 **Adilet — Technologies · затем Sanzhar**

"We built it with React, Node.js and SQLite because it runs in any browser with zero

installation. Our database stores users, ingredients, recipes and a photo diary in seven

tables — and the ingredient matching itself runs in SQL."

«React + Node + SQLite — работает в любом браузере без установки. В БД семь таблиц:

пользователи, ингредиенты, рецепты, дневник; матчинг — в SQL».)

1:30–1:50 **Sanzhar — Security + Testing**

"...and we protect users by hashing passwords with scrypt, signing session cookies, and rate-limiting brute-force attacks. We tested the product with real users — three UX problems found, three fixed. And our database audit found two ingredients with zero recipes — we added ten recipes, and dead coverage dropped to zero."

«...защита: scrypt-хэши паролей, подписанные куки, рейт-лимит. Тестирование: три UX-проблемы — исправлены. Аудит БД: два ингредиента без рецептов — добавили десять, покрытие полное».)

1:50–2:00 **Artur — Plans + Handover**

"Next, we plan a fridge scanner with real computer vision — a working prototype is already in the app. Each of us owned a layer of this product. Now — let us show you how it works."

«Дальше — сканер холодильника с настоящим CV, прототип уже в приложении. У каждого

своя часть. Показываем!»)

2 · Live Demo — 2:00 (клик по клику)

> **До прихода комиссии:** сайт https://mealmate-29tt.onrender.com уже открыт (главная), вторая вкладка — localhost:3001 (резерв), третья — dbdiagram.io с ER, ролик готов к проигрыванию.

Время	Действие	Что говорить
0:00	Home (открыта)	"This is MealMate — live, deployed, not a mock-up. Everything is real data from our database."
0:10	Start cooking → тик **Potato, Onion, Eggs**	"I tick what I have — matches update instantly. Salt and oil are staples — already counted."
0:25	Показать два списка	"Cook right now: Crispy Fried Potatoes, 2 of 2. Almost there — with the exact missing items."
0:40	Open recipe → прокрутить шаги	"Detailed steps, KBJU per serving, every ingredient marked: green = have, red = buy."
1:00	ҚАЗ в хедере	"One click — full localization, including the cooking steps."
1:15	Scan products (баннер на /kitchen)	"Our newest feature: a fridge scanner. The camera is real; recognition is simulated — our honest next step." (честность = баллы)
1:35	Community	"Cooked dishes are shared with the whole university."
1:50	Закрытие	"Runs on a free deployment — the link is on our poster. Source code on GitHub. Questions?"

> **Если что-то сломалось:** Render висит → вкладка localhost:3001 (start-site.bat). Нет интернета вообще → ролик MealMate-promo.mp4 + рассказ по постеру. Камера в сканере не разрешена → "On a phone it works — the pipeline is the same" и закрыть модалку.

3 · CHEAT — Artur (Project Lead)

Мои строки

"76 kilograms per person per year — UNEP 2021." · "Every tester found a recipe in under two minutes." · "Next: a fridge scanner with real computer vision — a prototype is already live."

Ответы комиссии

How do you know the problem is real? Three sources: UNEP Food Waste Index 2021 (76 kg/person/year, Kazakhstan); a 2021 study of 244 students across five Almaty universities linking food security to well-being; and our own tests — three external testers all completed the core task.

What did YOU personally do? I owned the problem: research, evidence, priorities, acceptance. After UX tests I pushed three concrete fixes — clearer ingredient selection, an explicit finish state, visible navigation. I kept scope honest: we cut features instead of claiming unfinished ones.

How are you different from Supercook / Yummly? Local cuisine first-class (beshbarmak, kuurdak, baursaki); the honest two-list result with exact missing items; full Kazakh and Russian interface. Global services barely cover any of this.

Business side? Today — a student project on free hosting. The pattern that can earn: delivery integration ("2 ingredients away from these dishes") and university partnerships for dorms. Honest: monetization is not built yet.

Emergency "Outside my layer — my teammate owns it, and we would verify with data first." · Demo breaks → backup tab localhost:3001, or the promo video.

4 · CHEAT — Arsen (Frontend)

Мои строки

"You tick your ingredients — it instantly splits 68 recipes into 'cook right now' and 'almost there'." · "Kazakh cuisine is first-class — and it works in three languages." · "The scanner camera is real; the AI recognition is simulated — our honest next step."

Ответы комиссии

What did YOU personally do? The entire client layer: a hand-written CSS design system (no UI kit), all pages — picker with live matching, catalog, recipes, diary, community — and full trilingual localization including cooking steps. Plus the fixes our tests exposed: a pantry sync race, a quick-add bug, category overflow.

Why React, not plain HTML or mobile? The picker is one reactive state machine — ticking a chip must update matches instantly. Web over mobile because zero installation is our core requirement; a Capacitor wrapper would give app stores later without a rewrite.

How does localization work? Two layers. Content: localized DB columns (title_ru, name_kk, steps too) — the API picks with ?lang, English fallback. Interface: one shared dictionary. Units and phrases — "a pinch", "ас қасық" — localized separately.

What is simulated or unfinished? Honestly: scanner recognition is client-side simulated (camera and UI are real); guest diary photos reset on hosting restarts (ephemeral disk); password-reset emails are shown on screen — no mail service yet.

How are images handled? In the browser before upload: canvas downscale to 1280 px, JPEG quality 0.82 — a 5 MB phone photo becomes ~200 KB. Server re-validates format and size.

5 · CHEAT — Adilet (Database)

Мои строки

"We built it with React, Node.js and SQLite — zero installation." · "Seven tables — and the ingredient matching itself runs in SQL."

Ответы комиссии

Why SQLite and not PostgreSQL? Honest engineering: our load is a university demo — SQLite gives real persistence with zero administration, and the driver is built into Node. All SQL sits behind one thin module (db.js), so moving to PostgreSQL means rewriting that one file. We already avoid SQLite-only features.

Show the ER model. What would you change live? Seven entities: users, ingredients, recipes, two M:N junctions, instruction blocks, photo diary — plus reset tokens. Live change example: add **servings_scale** to recipe_ingredients so users can cook for three instead of two — one column, no schema break. The editable source is docs/er-diagram.dbml — paste at dbdiagram.io.

Where is the matching logic — DB or code? In SQL. Two CTEs: req counts required ingredients per recipe (non-optional, non-staple); got counts the user's overlap. missing = total − have. Zero → "cook now", 1–2 → "almost there". Filters run in the same query.

Relevance guarantee — what if I tick only bacon? Two guards: have ≥ 1 — a recipe appears only if at least one required ingredient is selected; and with a tiny selection the "almost" threshold adapts — otherwise one bacon tick would hide carbonara because three items are missing.

Data integrity? FK with ON DELETE CASCADE; UNIQUE e-mail; UNIQUE (recipe_id, step_number); indexes on junctions and diary; parameterized queries only.

6 · CHEAT — Sanzhar (Backend & Security)

Мои строки

"…we protect users by hashing passwords with scrypt, signing session cookies, and rate-limiting brute-force attacks." · "Testing: three UX problems found, three fixed. Database audit: two ingredients with zero recipes — ten recipes added, coverage zero gaps."

Ответы комиссии

What did YOU personally do? Every API endpoint — ingredients, matching, auth, pantry, diary, community feed. The security stack: scrypt hashing, HMAC-signed cookies, headers, rate limits, input validation. And the content pipeline: 68 recipes with nutrition and three-language steps, seeded into the database.

How are passwords stored? scrypt — a memory-hard KDF built into Node — with a unique random salt per user, compared with a timing-safe check. A database leak gives salted hashes; brute-forcing each costs heavy memory and time.

Sessions without a login server? Signed cookie: user id + expiry, HMAC-SHA256 with a server secret. httpOnly (JS cannot read it) + SameSite=Lax. Guests use the same mechanics with a guest row; registering upgrades that row — their pantry carries over.

Demonstrate one security scenario. Brute force: rapid wrong passwords at /api/auth/login — first forty get 401, request forty-one gets HTTP 429 for the rest of the window. scrypt makes each offline guess cost ~100 ms of CPU. Safe scenario: our own deployment, test account — no third-party systems touched.

How did you use AI and verify it? Disclosed in every assignment. AI wrote code, seed content and the coverage workbook. Verified by running: recipes reviewed against real cooking logic, endpoints tested with live requests, the Assignment-4 numbers come from database queries I can re-run here. Honest gaps — simulated scanner AI — are disclosed inside the product.

What if the database leaks? Worst case: usernames, e-mails, salted scrypt hashes and public recipe data. No plaintext passwords, no payment data. Sessions survive — they are signed with a server-side secret, not DB content.

7 · Checklist дня — 6 ноября

Когда	Что
Сегодня вечером	Спич с таймером ×2 · демо на живом сайте от начала до конца · ноутбук на зарядку · шпаргалки распечатаны / скачаны на телефон
Утром	Проверить https://mealmate-29tt.onrender.com · проверить резерв: start-site.bat → localhost:3001 · ролик MealMate-promo.mp4 воспроизводится
На станции (до 14:45)	Ноутбук открыт на главной · вкладка 2: localhost:3001 · вкладка 3: dbdiagram.io с er-diagram.dbml · папка docs открыта · постер на столе
После защиты	Поблагодарить комиссию · сфотографировать постер · отметить фидбек для End-term

Файл: docs/finals (HTML + PDF). Источник истины по проекту: docs/DEFENSE.md · README.md. MealMate © CS-2618 — Samarin · Kaniyazov · Sadybek · Temirtas.